Contents

Technical Architecture & Solution Presentation: Advidify Platform

Lead System Architect Presentation to Enterprise Stakeholders

---

Executive Summary

Advidify (AI Video Crafters) is an enterprise-grade, high-touch custom video production platform. Rather than relying on unvetted, fully automated Generative AI outputs which often fail commercial quality and licensing standards, Advidify pioneeringly implements a **"Human-in-the-Loop" (HITL)** operational paradigm. The platform connects enterprise clients with skilled video, audio, and advertising professionals who utilize generative AI tools as leverage to create bespoke, high-end videos.

The system is built on a highly secure, high-performance, and decentralized microservice architecture. Engineered with a **Rust-based backend** ecosystem, a custom reverse-proxy gateway powered by Cloudflare's **Pingora** framework, and modern **React 19 / TypeScript** frontend applications, Advidify ensures near-zero latency, military-grade security, and robust resilience.

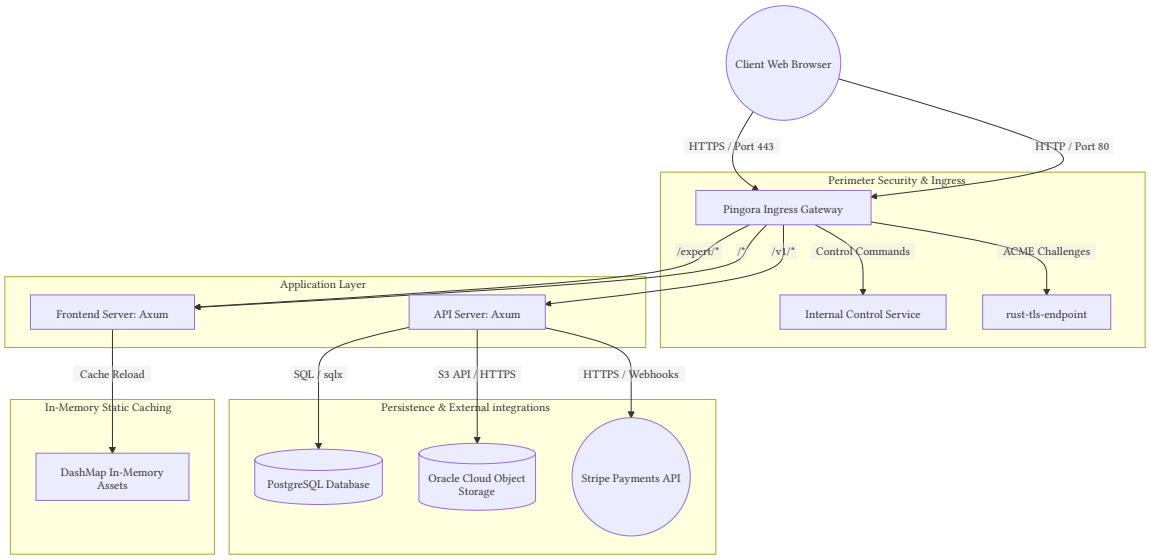
---

1. Architecture & System Design

1.1 Overarching System Architecture

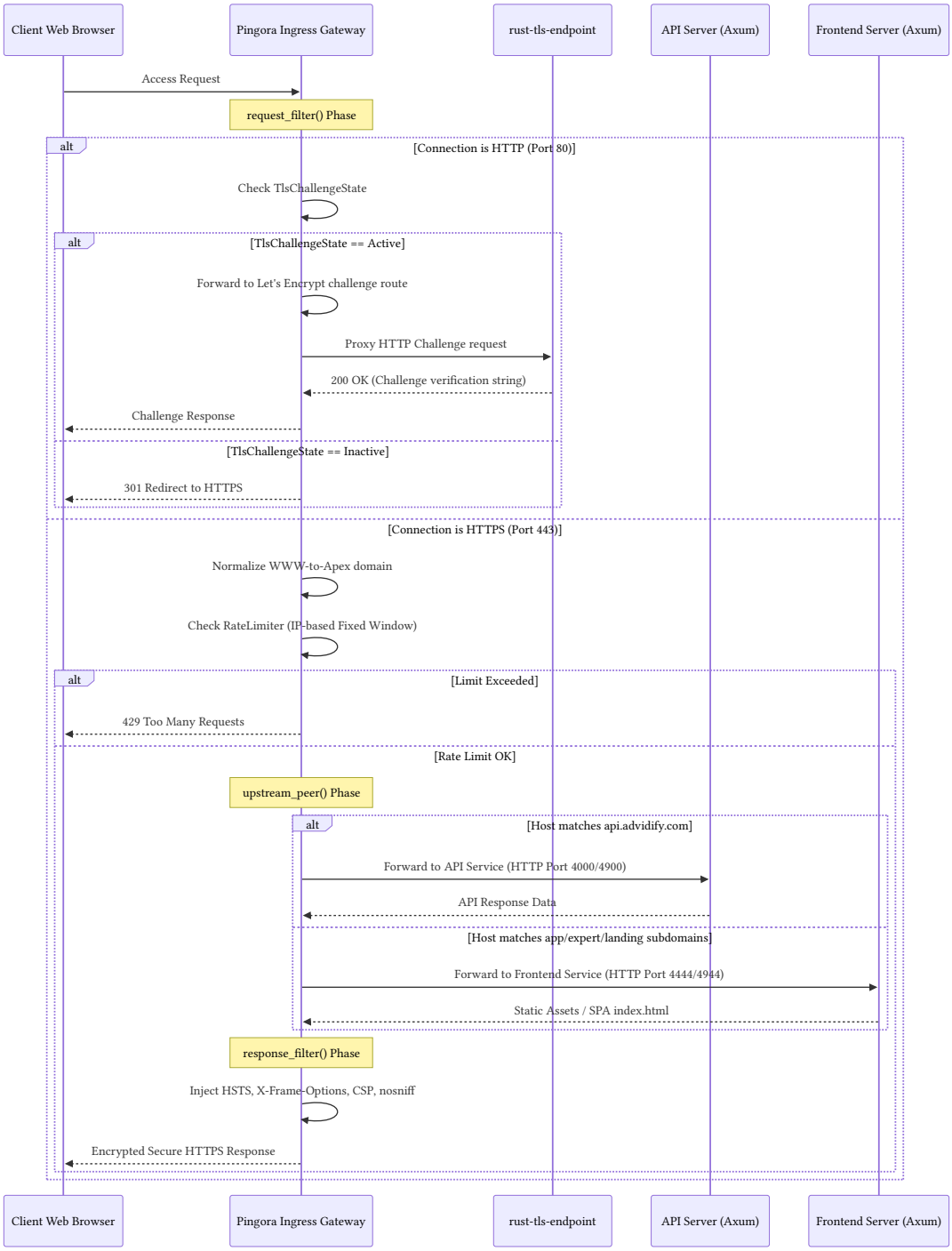
The Advidify platform separates ingress routing, static asset delivery, database migrations, and business logic processing into decoupled modules. All ingress traffic is terminated and processed

at the perimeter by the custom Pingora Gateway, which forwards requests internally to target Axum-based microservices.



1.2 Request Lifecycle & Routing Control Flow

All client communication undergoes a strict inspection sequence upon reaching the Pingora proxy. Below is the request routing, verification, and redirection sequence executed on each HTTP/HTTPS ingress handshake.

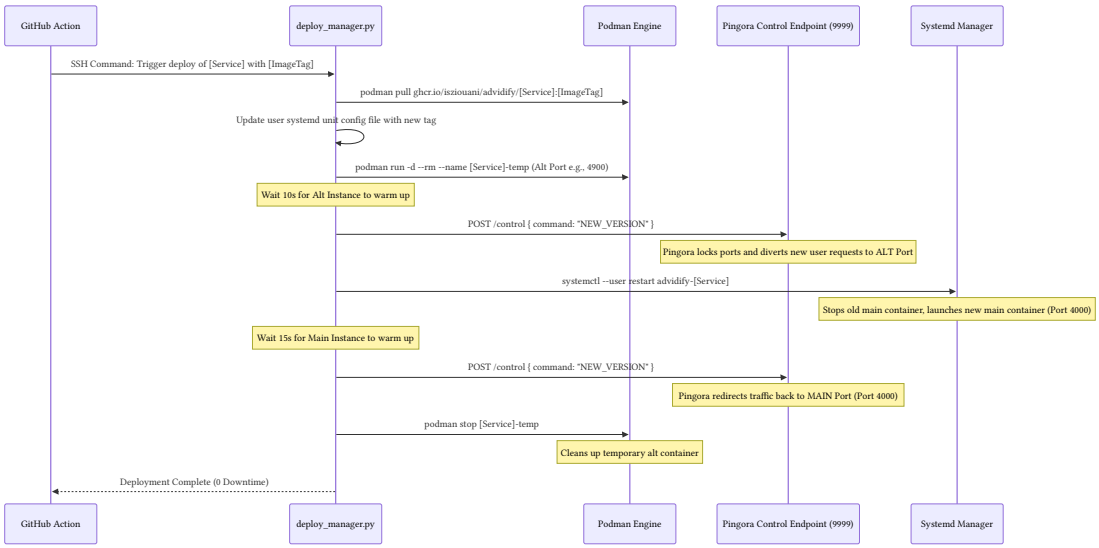


1.3 DevOps Pipeline: Zero-Downtime Blue-Green Deployment

Deployments are fully automated, tag-driven, and run-once orchestrated. When a new tag (e.g., `api-server-v1.0.0`) is pushed to GitHub, GitHub Actions triggers the deployment pipeline. Rather than performing a complex build on the production system, the workflow acts as an or-

chestrator, invoking a custom Python deployment manager (`deploy_manager.py`) which manages **Podman** containers.

A **Bridge Deployment Strategy** is utilized to toggle active traffic back and forth using Pingora's private control interface, ensuring true zero-downtime updates:



2. Technology Stack & Microservices Breakdown

2.1 Technology Stack Inventory

Component	Implemented Technology	Engineering Rationale
Core Backend	Rust 1.85.0	Extreme memory safety (no garbage collection overhead), compiled performance, and strict concurrency primitives.
API Framework	Axum (Tokio Ecosystem)	Asynchronous, routing-centric web framework utilizing native macro-free types and direct compatibility with standard Tower middleware.
Perimeter Ingress	Cloudflare Pingora	Custom high-performance, asynchronous Rust proxy engine. Provides programmatic control over request filtering, load toggling, and SSL termination.
Database ORM	SQLx (PostgreSQL driver)	Compile-time verified SQL queries against a live schema database, preserving the performance of raw SQL without raw string unsafety.
Client Frontend	React 19 (TypeScript)	Strict component-driven, statically typed views. Decoupled client and expert contexts optimize browser performance and bundle sizes.
Frontend Styling	Tailwind CSS 4	Atomic utility engine providing modern CSS compilation, reducing layout footprint and rendering times.
Static Build/SEO	Prerender (Custom SSG script)	Pre-renders the public marketing site (`appfrontend/signout`) into clean static HTML during build time, ensuring sub-millisecond loads and perfect SEO indexability.
Storage Engine	Oracle Cloud Infrastructure (OCI)	Cost-efficient enterprise storage served via an S3-compatible API wrapper crate.
Telemetry	OpenTelemetry, tracing-opentelemetry	Unified observability stack sending metric pipelines (CPU, Memory via `sysinfo`) and tracing spans to Otel-compatible collectors.
Containerization	Podman (rootless mode)	Daemonless, rootless container execution, ensuring strict process isolation and reduced attack vectors on the host operating system.

2.2 Microservices & Core Modules Analysis

2.2.1 `pingora-gateway`

- **Role**: The perimeter ingress point. Terminating TLS connections, managing WWW-to-Apex redirection, enforcing rate limiting, and executing blue-green deployment toggles.
- **Technology**: Pingora Framework, `tokio` async runtime, `dashmap` lock-free concurrent map.
- **Engineering Rationale**: Pingora out-performs traditional Nginx configurations under heavy concurrent request loops. Implementing rate limiting and gateway toggles inside Rust gives developers programmatic, thread-safe access to network pipelines without writing custom C/Lua modules.

2.2.2 `api-server`

- **Role**: The core business brain. Orchestrates WebAuthn passkey flows, project workflow status transitions, file upload validation, and database operations.
- **Technology**: Axum, SQLx, `webauthn-rs`, `stripe` client.
- **Architecture**: Mixed Hexagonal & Pyramid pattern:
  - `api/` (driving adapters): HTTP routing, validation, and JSON extracting.
  - `domain/` (core logic): Pure business models, workflow states, and rules.
  - `driven/` (driven adapters): Repository implementations talking to PostgreSQL (`sqlx`) and storage (`oci_s3`).
- **Engineering Rationale**: The separation of ports and adapters isolates the core billing and project status code from HTTP delivery details, making it highly testable and robust to database or storage vendor swaps.

2.2.3 `frontend-server`

- **Role**: An in-memory static file server serving compiled React SPA applications.
- **Technology**: Axum, `DashMap`, `brotli` pre-compression.
- **Engineering Rationale**: Traditional static file serving incurs disk I/O bottlenecks. Upon initialization, the frontend server reads the `.html`, `.js`, and `.css` assets, compresses them using Brotli, and stores them in memory inside a sharded `DashMap`. It serves clients with zero disk reads, checking the `Accept-Encoding` header for Brotli support, resulting in sub-millisecond response times.

2.2.4 `rust-tls-endpoint`

- **Role**: Automates certificate generation by answering Let's Encrypt HTTP-01 challenges.
- **Technology**: Axum (port 8000).
- **Engineering Rationale**: Because the Pingora Gateway enforces HTTPS redirection globally, automatic ACME renewal tools (like `certbot`) fail. When certbot is invoked, Pingora toggles `TlsChallengeState` to `Active`, routes HTTP requests for `.well-known/acme-challenge/*` to this microservice, reads the verification token from disk, validates it with Let's Encrypt, and toggles HTTPS enforcement back to normal.

2.2.5 `maintenance`

- **Role**: A lightweight standby server serving a "Scheduled Maintenance" message.
- **Technology**: Axum (port 5599).
- **Engineering Rationale**: During database schema migrations or system upgrades, Pingora can toggle all internal upstreams to point to this standby service, avoiding ugly browser-default connection errors.

2.2.6 `tooling`

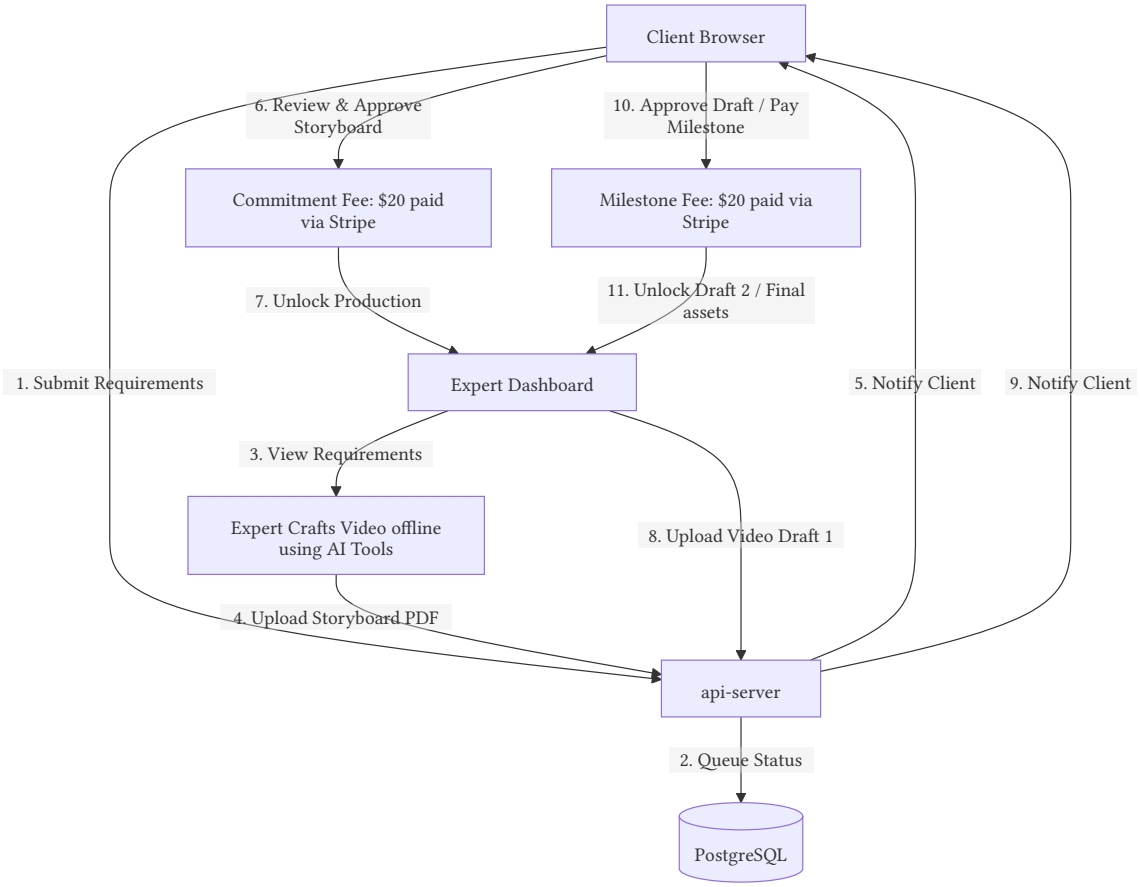
- **Role**: Database migration and seeding utility.
- **Technology**: SQLx CLI runner, `base64` decoders.
- **Engineering Rationale**: Keeps schema updates version-controlled and executes statement-by-statement SQL parsing, ensuring transactional database setup.

---

3. AI Integration & Logic

3.1 Human-in-the-Loop (HITL) Architectural Design

Advidify's core design explicitly rejects automated, low-quality AI asset generation in favor of a hybrid approach. The AI logic is structured to augment human creators rather than bypass them.



3.2 The Scaffolded 'AI Balance System' & Credits Engine

The system is built to support a future **usage-based token/credits economy** alongside its current monetary payment system. The database is prepared with specific schemas: * `SUBSCRIPTIONS`: Tracks client tiers, monthly credit allocations, and subscription dates. * `PURCHASED_CREDITS`: Manages pay-as-you-go token bundles, purchase dates, and expiration intervals. * `FEATURE_CONSUMPTION`: Tracks programmatic feature calls and the amount of resources consumed.

Inside the api-server adapter code (`update_credits_token.rs`): * The credit deduction algorithms (the `deduct_credits` logic) are fully scaffolded, sorting purchased credits by age (oldest first) to deduct tokens sequentially before reverting to active subscriptions or applying generosity allowances. * **Honest Implementation State**: To align with our current operational focus on usage-based Milestone Payments, the programmatic token-deduction logic (`deduct_credits`) is temporarily commented out. The active system gates user actions strictly using the `check_roles` function. The roles map directly to verification states, enforcing Stripe milestones ($20 Commitment Fee for the storyboard, $20 Milestone Fees per video draft segment, and revision tracking allowing 2 free revisions before billing $20 per extra revision).

---

4. Enterprise-Grade Standards & Compliance

4.1 Production Security & Hardening Protocols

Security is integrated at every tier of the application stack, from the host operating system up to the browser runtime:

1. **Phishing-Resistant Passwordless Authentication (WebAuthn)**: Advidify completely eliminates passwords. By integrating the FIDO2 standard using `webauthn-rs`, users register and authenticate using passkeys (face recognition, fingerprint scanners, or physical hardware tokens).
   - No passwords or hashes are transmitted or saved in PostgreSQL.
   - Mitigates credential-stuffing, SQL-injection database dumps, and phishing attack vectors.
   - Authentication requests verify credentials cryptographically using unique public-private keypairs.
2. **In-Memory Cryptographic Secrets Vault**: The `Vault` utility manages encryption keys in memory. Database secrets (such as API keys or configuration strings) are encrypted at rest using AES-GCM algorithms. The decryption keys are parsed from environment strings at server boot and stored inside a thread-safe `DashMap` in memory, ensuring database-layer compromises do not expose downstream integration credentials.
3. **Perimeter Network Shielding**:
   - **Ingress Rate Limiting**: Programmed into the Pingora gateway, rejecting traffic spikes on a per-IP fixed-window filter.

- **Strict Security Headers**: Pingora injects HSTS (`max-age=31536000`), `X-Frame-Options: DENY` (anti-clickjacking), `X-Content-Type-Options: nosniff` (anti-MIME-sniffing), and custom Content Security Policies (CSP) into all outgoing HTTP headers.
- **Disposable Email Blocklist**: During sign-up, the system matches user emails against a massive blocklist file containing over 25,000 disposable domains to mitigate spam and fake account creation.

4. **OS & Kernel-Level Security Hardening**: As documented in our production environments (`production.md`):

- **Locked Down Ingress Ports**: System access is limited to port 2222 (hardened SSH key-only access), port 80, and port 443. All other ports are locked by default via **UFW**.
- **Fail2ban Integration**: Protects against SSH brute-forcing and unauthorized PostgreSQL connection attempts.
- **AppArmor Profiles**: Every microservice binary (e.g., `api-server`, `frontend-server`) is locked down with strict AppArmor profiles, preventing them from writing to arbitrary directories or spawning unsanctioned processes.
- **Kernel hardening (sysctl)**: The kernel is configured to ignore ICMP redirects, block SYN-flood attacks (`tcp_syncookies = 1`), and enforce ASLR security.

4.2 Scalability & Concurrency Model

The platform handles high-concurrency demands by removing blockages in the thread pool: * **Lock-Free DashMaps**: Instead of using global mutex locks (which block reader threads during updates), our caching layers rely on `DashMap` (sharded concurrent hashmap). * **Asynchronous Task Offloading**: Slow I/O tasks (such as sending emails or initiating Stripe customers) are offloaded onto thread-safe `tokio::mpsc` async channels. HTTP requests immediately return a response, and background worker loops handle the execution asynchronously. * **Zero Disk I/O Static Serving**: serving files from memory saves CPU cycles and disk access wait times.

4.3 Data Privacy, Ethical AI & Compliance

Advidify is designed to align with strict global data privacy regulations (such as GDPR and CCPA): 1. **Privacy-First Data Architecture**: The platform collects and stores minimal user data. The database schema separates client metadata, billing logs, and draft assets. All client files are stored on secure cloud object storage (OCI) with timed access URLs. 2. **Data Erasure Compliance**: Upon project completion, all project assets, drafts, and metadata can be deleted, leaving only historical billing rows required for corporate tax compliance. 3. **Ethical AI & Licensing Safety**: By avoiding fully automated generation, Advidify ensures that all output videos undergo strict human review. Every video draft is validated by human experts to ensure: - Compliance with third-party generative AI tool licenses. - Prevention of copyright infringements. - Clear disclosure of AI tool usage to the client. - Strict policies prohibiting deepfakes or celebrity likeness generation without authorization.